Simulation Mentoring

Simulation Developing Examples & Proofs

Written by. 홍준서, 백재원

March 6, 2026

Projectile Simulation

There is a projectile which is launched with

$$\vec{r_0} = (x_0, y_0)$$

$$\vec{v_0} = (v_{0x}, v_{0y})$$

The projectile is affected by gravity:

$$\vec{F_g} = (0, -mg)$$

The projectile is also affected by air resistance:

$$\vec{F_r} = (-kv_x, -kv_y)$$

There are some lines that the projectile can collide with. Each line is represented by two points:

$$p_i = (\vec{p_{i1}}, \vec{p_{i2}})$$

And there is constraint that

$$\forall i, \vec{d_i} \parallel \hat{x} \vee \vec{d_i} \parallel \hat{y}$$

$$\forall i, \vec{p_{i2}} = \vec{p}_{(i+1)1}$$

where $\vec{d_i} = \vec{p_{i2}} - \vec{p_{i1}}$.

Proof that the projectile cannot collide with a vertical edge

We consider a projectile with initial position

$$\vec{r}_0 = (x_0, y_0)$$

and initial velocity

$$\vec{v}_0 = (v_{0x}, v_{0y})$$

The forces acting on the projectile are gravity

$$\vec{F}_g = (0, -mg)$$

and linear air resistance

$$\vec{F}_r = (-kv_x, -kv_y)$$

We prove that the projectile cannot collide with any edge parallel to the y axis.

Step 1: Equation of motion in the x direction

By Newton's second law,

$$m\frac{dv_x}{dt} = -kv_x$$

This is a separable differential equation:

$$\frac{dv_x}{v_x} = -\frac{k}{m}dt$$

Integrating:

$$\ln|v_x| = -\frac{k}{m}t + C$$

Thus,

$$v_x(t) = v_{0x}e^{-\frac{k}{m}t}$$

Step 2: Position function $x(t)$

Integrating velocity:

$$x(t) = x_0 + \int_0^t v_x(s)ds$$

$$x(t) = x_0 + \frac{m}{k} v_{0x} \left(1 - e^{-\frac{k}{m}t}\right)$$

Step 3: Monotonicity of $x(t)$

Since

$$e^{-\frac{k}{m}t} > 0 \quad \forall t$$

we have

$$v_x(t) = v_{0x} e^{-\frac{k}{m}t}$$

Therefore the sign of $v_x(t)$ is constant:

$$\text{sign}(v_x(t)) = \text{sign}(v_{0x})$$

Thus,

$$\frac{dx}{dt} = v_x(t)$$

never changes sign.
Therefore,

$$x(t)$$

is a strictly monotonic function.

Step 4: Vertical edge definition

A vertical edge is defined by

$$x = c$$

for some constant c.
A collision would require two distinct times $t_1 \neq t_2$ such that

$$x(t_1) = x(t_2) = c$$

Step 5: Application of the Intermediate Value Theorem

The function $x(t)$ is continuous.
Since it is strictly monotonic, it is injective.
Therefore,

$$x(t_1) = x(t_2) \implies t_1 = t_2$$

Thus, there cannot exist two distinct times where the projectile has the same x coordinate.

Conclusion

Since a collision with a vertical edge would require the projectile to reach the same x coordinate more than once, and since $x(t)$ is strictly monotonic, this is impossible.

Therefore,

$$\boxed{\text{The projectile cannot collide with any edge parallel to the } y \text{ axis.}}$$

Numerical Integration Convergence

In physics simulations, we often use simple update rules to approximate continuous motion. This section explains that as the time step $dt \to 0$, these numerical updates converge to the true analytical physical solutions.

Case 1: Drag Force (Velocity Convergence)

Mathematical Model: Consider a particle subject to a drag force proportional to its velocity:

$$F = -kv \implies m\frac{dv}{dt} = -kv \implies \frac{dv}{dt} = -\frac{k}{m}v$$

The analytical solution is $v(t) = v_0 e^{-(k/m)t}$.

Discretized Update Rule: In a simulation, we update the velocity using the rule:

$$v+ = -(k/m)v \cdot dt \implies v_{n+1} = v_n\left(1 - \frac{k}{m}dt\right)$$

Convergence Explanation: After n steps ($t = n \cdot dt$), the velocity is:

$$v_n = v_0\left(1 - \frac{k}{m}dt\right)^n = v_0\left(1 - \frac{k}{m}\frac{t}{n}\right)^n$$

Using the definition of the exponential function $\lim_{n\to\infty}(1 - \frac{a}{n})^n = e^{-a}$:

$$\lim_{dt\to 0} v_n = v_0 e^{-(k/m)t}$$

Thus, the simulation result converges to the actual physical solution as $dt \to 0$.

Case 2: Integrating Velocity to Position

Mathematical Model: Given a velocity function $v(t)$, the continuous equation is $dx/dt = v(t)$, which yields:

$$x(t) = x_0 + \int_0^t v(s)ds$$

Discretized Update Rule: The simulation repeatedly applies:

$$x+ = v \cdot dt \implies x_{n+1} = x_n + v(t_n)dt$$

Convergence Explanation: Summing the updates yields:

$$x_n = x_0 + \sum_{k=0}^{n-1} v(t_k)dt$$

This is exactly the **Riemann Sum** of the velocity function. As $dt \to 0$, the Riemann sum converges to the definite integral:

$$\lim_{dt\to 0} x_n = x_0 + \int_0^t v(s)ds = x(t)$$

Case 3: Full Trajectory (Acceleration to Position)

Mathematical Model: Given an acceleration $a(t)$, the system is:

$$\frac{dv}{dt} = a(t), \quad \frac{dx}{dt} = v(t)$$

Discretized Update Rule: The simulation updates both velocity and position:

$$v+ = a \cdot dt, \quad x+ = v \cdot dt$$

Convergence Explanation: As shown in Case 2, the velocity update converges to $v(t) = v_0 + \int a(t)dt$. Then, the position update, which uses these converged velocity values, also converges to the integral of $v(t)$. Therefore, the entire chain $a \to v \to x$ converges to the physical trajectory as $dt \to 0$.

Control Flow Performance: If-Else vs Switch

In high-performance simulations, the choice of control flow structures can significantly impact execution time. This section compares the `if-else` chain and the `switch` statement through a particle simulation where each particle's behavior depends on its type.

Simulation Scenario

Consider a system of N particles where each particle belongs to one of several types:

$$\text{type} \in \{\text{TYPE_A, TYPE_B, TYPE_C, TYPE_D, TYPE_E}\}$$

For each particle, the acceleration $\vec{a}$ is assigned based on its type. We simulate $1,000,000$ particles over $1,000$ steps with a time step Δt, comparing the time taken by two different implementation styles for type-based acceleration assignment.

Mathematical Model and Update Rule

The state of each particle is updated using the standard Euler method:

$$\vec{v}_{n+1} = \vec{v}_n + \vec{a}(\text{type})\Delta t$$

$$\vec{x}_{n+1} = \vec{x}_n + \vec{v}_{n+1}\Delta t$$

The function $\vec{a}(\text{type})$ is the bottleneck of this simulation as it is called 10^9 times.

Implementation Comparison

1. If-Else Chain: The `if-else` approach checks types sequentially:

```
if (type == A) return 0.1;
else if (type == B) return 0.2;
...
```

In the worst case, the CPU performs multiple comparisons for a single particle, leading to $O(n)$ branching complexity.

2. Switch Statement: The `switch` approach provides a structured way to handle multiple discrete values:

```
switch (type) {
    case A: return 0.1;
    case B: return 0.2;
    ...
}
```

Simulation Interpretation: Jump Tables

Modern compilers often optimize a `switch` statement into a **Jump Table**. Instead of sequential comparisons, the CPU uses the `state` variable as an index into a table of memory addresses:

$$\text{TargetAddress} = \text{TableBase} + (\text{state} \times \text{EntrySize})$$

This allows the CPU to branch to the correct code block in $O(1)$ time. In contrast, an `if-else` chain performs $O(n)$ comparisons, which can lead to frequent branch mispredictions and pipeline stalls, especially when state transitions are frequent or random.

Summary

The simulation in `p2_jump_table.cpp` demonstrates that for discrete state-based branching, the `switch` statement is generally more efficient due to its ability to be compiled into a jump table, providing constant-time branch resolution regardless of the number of states.

Circular Motion Simulation

Consider a particle moving in a circular path with radius R. Its position and velocity are

$$\vec{r} = (x, y), \quad \vec{v} = (v_x, v_y)$$

subject to a centripetal force

$$\vec{F_c} = -m\omega^2\vec{r}$$

where $a = -\omega^2 x$ in each dimension. This simulation explores the stability of different update sequences.

Time Integration: Semi-Implicit Euler Method

In `p3_circle.cpp`, the simulation uses the following update sequence:

$$\vec{v}_{n+1} = \vec{v}_n + \vec{a}(\vec{x}_n)dt$$

$$\vec{x}_{n+1} = \vec{x}_n + \vec{v}_{n+1}dt$$

This sequence is a simple yet powerful way to maintain stability in physical simulations.

Why Velocity is Updated First: Time Grid Perspective

The reason for updating velocity before position can be understood through the relationship between velocity and the time grid.

- **Time Offset (Staggered Grid):** Ideally, to calculate the next position x_{n+1}, we should use a velocity that represents the average motion over the interval $[t_n, t_{n+1}]$. By updating v to v_{n+1} first and then using it for x, we effectively use a velocity that is "ahead" of the current position. This creates a time offset between x and v, making the simulation behave as if velocity is evaluated at the midpoint of the time step ($t_{n+1/2}$).

- **Feedback in Position-Dependent Forces:** In most physical systems, the force (and thus acceleration $\vec{a}$) is a function of position, $\vec{a} = \vec{a}(\vec{x})$. When we update $\vec{v}$ first using $\vec{a}(\vec{x}_n)$, the subsequent position update $\vec{x}_{n+1}$ is driven by a velocity that already "knows" the force acting at the previous position. This prevents the system from overshooting.

- **Energy Stability:** In circular motion, using the old velocity $\vec{v}_n$ (Explicit Euler) causes the particle to move linearly for the duration of dt, resulting in an ever-increasing radius and orbital energy. Using the updated velocity $\vec{v}_{n+1}$ corrects this by "pulling" the position update back toward the center, keeping the orbital energy bounded and the trajectory stable.

Summary of Update Rules

1. Compute acceleration $\vec{a}_n$ from the current position $\vec{x}_n$. 2. Update velocity: $\vec{v}_{n+1} = \vec{v}_n + \vec{a}_n dt$. 3. Update position using the *new* velocity: $\vec{x}_{n+1} = \vec{x}_n + \vec{v}_{n+1}dt$.

Grid-Based Heat Conduction Simulation

This simulation demonstrates how heat diffuses over a 1D spatial grid using numerical discretization of the heat equation.

Mathematical Model

The evolution of temperature $T(x,t)$ is governed by the heat equation:

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2}$$

where α is the thermal diffusivity.

Spatial Discretization (Finite Difference)

To solve this on a computer, we divide space into a grid of points T_i with spacing h. The second-order spatial derivative is approximated as:

$$\frac{\partial^2 T}{\partial x^2} \approx \frac{T_{i+1} - 2T_i + T_{i-1}}{h^2}$$

This allows heat to flow between adjacent cells based on their temperature difference.

Discretized Update Rule

The simulation in `p3_grid.cpp` employs a two-step update (Heun's method) for better stability:
1. Predictor step (Simple Euler):

$$\tilde{T}_i = T_i^n + \alpha dt \frac{T_{i+1}^n - 2T_i^n + T_{i-1}^n}{h^2}$$

2. Corrector step:

$$T_i^{n+1} = T_i^n + \frac{\alpha dt}{2}\left[\left(\frac{\partial^2 T}{\partial x^2}\right)_{T^n} + \left(\frac{\partial^2 T}{\partial x^2}\right)_{\tilde{T}}\right]$$

While the basic update rule $T_i^{new} = T_i + \alpha dt(\Delta T/h^2)$ is the foundation, this predictor-corrector approach provides a more accurate approximation of the continuous diffusion process.

Simulation Performance Experiments

This section explores various implementation strategies that affect the performance of large-scale simulations, focusing on branching, memory layout, and memory allocation.

Constraint Handling: If-Else vs Ternary Operator

Simulation Scenario: Particles are confined within a 1D boundary $[-B, B]$. When a particle exceeds the boundary, its position is clamped and its velocity is reversed.

Implementation Comparison: The simulation in `p4_branch.cpp` compares two ways to enforce these constraints:

- **If-Else:** Uses standard conditional blocks.
- **Ternary Operator:** Uses inline conditional expressions.

Performance Interpretation: While logically equivalent, compilers can often optimize ternary operators into **conditional move (CMOV)** instructions. Unlike jumps, CMOV instructions avoid the overhead of branch prediction, leading to more stable performance when boundary collisions are unpredictable.

Memory Layout: AoS vs SoA

Numerical simulations often process massive amounts of data. The way this data is arranged in memory significantly impacts performance due to CPU cache behavior.

AoS (Array of Structures):

$$\text{struct Particle } \{ \text{ float x, y, z, vx, vy, vz; } \}; \quad \text{Particle p[N];}$$

Each particle's attributes are stored together. This is intuitive but can be inefficient when only a subset of data (e.g., only positions) is needed, as the CPU still loads the irrelevant velocity data into the cache.

SoA (Structure of Arrays):

$$\text{float x[N], y[N], z[N], vx[N], vy[N], vz[N];}$$

Each attribute is stored in its own contiguous array.

Performance Interpretation: The SoA layout is generally preferred in simulation kernels for two reasons:

1. **Cache Locality:** When updating only positions, the CPU loads a cache line entirely filled with position data. This maximizes cache utilization and reduces memory bandwidth waste.
2. **SIMD Utilization:** Contiguous attribute arrays allow the compiler to use **SIMD (Single Instruction, Multiple Data)** instructions, such as AVX-512, to process multiple particles (e.g., 8 or 16) in a single CPU cycle.

Memory Allocation: Stack vs Heap

Simulation Scenario: Large arrays of particles are allocated to measure the overhead of memory management.

Implementation Comparison:

- **Stack Allocation:** `Particle p[N];` (Size must be known at compile time or be small).
- **Heap Allocation:** `std::vector<Particle> p(N);` (Dynamic size).

Performance Interpretation: Stack allocation is nearly instantaneous as it only involves moving the stack pointer. Heap allocation requires searching for available blocks in the free store, which involves significant operating system overhead. In high-frequency simulations, frequent heap allocations can become a major bottleneck.

Conclusion

Optimizing simulation performance requires an understanding of how code translates to hardware operations. Minimizing branches, ensuring cache-friendly memory layouts (SoA), and preferring efficient allocation strategies are key to handling millions of particles in real-time.

Taylor Series Simulation

Numerical simulations often use Taylor series approximations to evaluate transcendental functions like e^x, $\sin(x)$, and $\cos(x)$. This section demonstrates how a function can be approximated by a polynomial of degree N.

Mathematical Model

The Taylor series of a smooth function $f(x)$ centered at a is defined as:

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!}(x-a)^n$$

For the exponential function e^x centered at $a = 0$ (also known as the Maclaurin series), we have:

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!} = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots$$

Simulation Scenario

The simulation in `p5_taylor.cpp` calculates the N-th partial sum $S_N(x)$:

$$S_N(x) = \sum_{n=0}^{N} \frac{x^n}{n!}$$

It compares $S_N(x)$ to the actual value obtained from `std::exp(x)` and measures the absolute error for increasing values of N.

Numerical Update Rule

To efficiently calculate each term without recomputing the entire factorial, the simulation uses an iterative approach:

$$\text{term}_n = \text{term}_{n-1} \times \frac{x}{n}$$

$$S_n = S_{n-1} + \text{term}_n$$

where $\text{term}_0 = 1$ and $S_0 = 1$.

Convergence Analysis

The Taylor series for e^x has an infinite radius of convergence, meaning:

$$\lim_{N\to\infty} S_N(x) = e^x \quad \forall x \in \mathbb{R}$$

The simulation results show that as N increases, the error decreases rapidly. For small x, the sum converges to machine precision within a few dozen terms.

Simulation Interpretation

While Taylor series are theoretically exact in the limit, numerical implementations must consider:

1. **Floating Point Precision:** Very small terms might eventually fall below the precision of a `double`.
2. **Computational Cost:** Higher values of N provide more accuracy but require more floating-point operations.

This simulation illustrates the fundamental trade-off between accuracy and computational resources in numerical approximation.

Graph-Based Structural System

There is a structure composed of a graph network

$$G = (V, E)$$

where V is the set of nodes and E is the set of edges.

Node properties

Each node $v \in V$ has:

- position: $\vec{r}_v = (x_v, y_v) \in \mathbb{R}^2$
- mass: $m_v \in \mathbb{R}$
- fixed state: $f_v \in \{\mathrm{T}, \mathrm{F}\}$

Edge properties

Each edge $e \in E$ has:

- rest length: $l_e \in \mathbb{R}$
- rate: $r_e \in \mathbb{R}$
- connected nodes:
$$n_e^{(\text{left})}, \quad n_e^{(\text{right})} \in V$$

Edges break when the normal force exceeds their strength.

Equation of Motion

We know

$$\vec{F} = M\vec{a}$$

Thus,

$$\vec{a} = M^{-1}\vec{F} \tag{1}$$

since M is diagonal and invertible.

Constraint Definition

Define the constraint vector

$$\vec{C}(\vec{r}) = \begin{bmatrix} C_1(\vec{r}) \\ C_2(\vec{r}) \\ \vdots \\ C_k(\vec{r}) \end{bmatrix}$$

Each constraint is defined as

$$C_e(\vec{r}) = l_e^2 - \left\| \vec{r}_{n_e^{(\text{left})}} - \vec{r}_{n_e^{(\text{right})}} \right\|^2$$

The system is in equilibrium when

$$\vec{C}(\vec{r}) = \vec{0}$$

Assume equilibrium at $\vec{r}_0$:

$$\vec{C}(\vec{r}_0) = \vec{0} \tag{2}$$

Let

$$\vec{r} = \vec{r}_0 + \vec{v}dt$$

Linear Approximation

Using multivariable Taylor expansion:

$$d\vec{C}(\vec{r}) = \frac{\partial \vec{C}}{\partial \vec{r}}\, d\vec{r}$$

Define the Jacobian

$$J = \frac{\partial \vec{C}}{\partial \vec{r}}$$

Then

$$\vec{C}(\vec{r} + d\vec{r}) \approx \vec{C}(\vec{r}) + Jd\vec{r}$$

For equilibrium,

$$\vec{C}(\vec{r}) + Jd\vec{r} = \vec{0} \tag{3}$$

Normal Force Representation

Normal force is aligned with constraint gradient:

$$\vec{N}_e \parallel \nabla C_e(\vec{r})$$

Thus total normal force:

$$\vec{N} = \sum_{e \in E} \lambda_e \nabla C_e(\vec{r})$$

Matrix form:

$$\vec{N} = J^T \vec{\lambda}$$

Acceleration with Constraint Force

$$\vec{a} = M^{-1}(\vec{F}_{ext} + \vec{N})$$

$$= M^{-1}\vec{F}_{ext} + M^{-1}J^T\vec{\lambda}$$

Define external acceleration

$$\vec{a}_{ext} = M^{-1}\vec{F}_{ext}$$

Then

$$\vec{a} = \vec{a}_{ext} + M^{-1}J^T\vec{\lambda}$$

Displacement due to Constraint Force

Constraint-only displacement:

$$d\vec{r} = \frac{1}{2}(\vec{a} - \vec{a}_{ext})dt^2$$

Thus,

$$d\vec{r} = \frac{1}{2}M^{-1}J^T\vec{\lambda}dt^2 \tag{4}$$

Solving for Lagrange Multiplier

Substitute (4) into (3):

$$\vec{C}(\vec{r}) = -Jd\vec{r}$$

$$= -\frac{1}{2}JM^{-1}J^T\vec{\lambda}dt^2$$

Thus,

$$\vec{\lambda} = -2\left(JM^{-1}J^T\right)^{-1}\vec{C}(\vec{r})dt^{-2}$$

Final Position Correction

Substitute into (4):

$$d\vec{r} = -M^{-1}J^T\left(JM^{-1}J^T\right)^{-1}\vec{C}(\vec{r})$$

Velocity correction:

$$\vec{v} \leftarrow \vec{v} + \frac{d\vec{r}}{dt}$$

Edge Breaking Condition

Normal force per edge:

$$\vec{N}_e = \lambda_e \nabla C_e(\vec{r})$$

This is proportional to normalized length change:

$$\propto \frac{1}{l_e}\left(l_e^2 - \|\vec{r}_i - \vec{r}_j\|^2\right)$$

Let length change be Δl_e.
Approximation:

$$\Delta l_e^2 \approx 2l_e\Delta l_e$$

Breaking condition:

$$\frac{\Delta l_e^2}{l_e} > kl_e$$

$$\frac{2l_e\Delta l_e}{l_e} > kl_e$$

$$\frac{\Delta l_e}{l_e} > \frac{k}{2} = k'$$

Thus edge breaks when strain exceeds threshold:

$$\boxed{\frac{\Delta l_e}{l_e} > k'}$$

Numerical Differentiation

Numerical differentiation is the process of approximating the derivative of a function using discrete values. This is essential in simulations where analytic derivatives are unknown or difficult to compute.

Forward Difference

The simplest approximation is the forward difference, derived directly from the definition of the derivative:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

Using Taylor expansion, $f(x+h) = f(x) + f'(x)h + \frac{1}{2}f''(\xi)h^2$. Substituting this reveals the truncation error:

$$\frac{f(x+h) - f(x)}{h} = f'(x) + O(h)$$

The forward difference is a first-order approximation.

Central Difference

A more accurate approach is the central difference, which uses values on both sides of x:

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$

Consider the Taylor expansions:

$$f(x+h) = f(x) + f'(x)h + \frac{1}{2}f''(x)h^2 + \frac{1}{6}f'''(\xi_1)h^3$$

$$f(x-h) = f(x) - f'(x)h + \frac{1}{2}f''(x)h^2 - \frac{1}{6}f'''(\xi_2)h^3$$

Subtracting the two equations:

$$f(x+h) - f(x-h) = 2f'(x)h + O(h^3)$$

$$\frac{f(x+h) - f(x-h)}{2h} = f'(x) + O(h^2)$$

The central difference is a second-order approximation, meaning the error decreases quadratically as $h \to 0$.

Comparison

In simulations, central difference is generally preferred over forward or backward difference because the $O(h^2)$ error term allows for much higher accuracy with the same step size h. However, it requires an extra function evaluation $f(x-h)$, which might not always be available (e.g., at the start of a simulation time series).